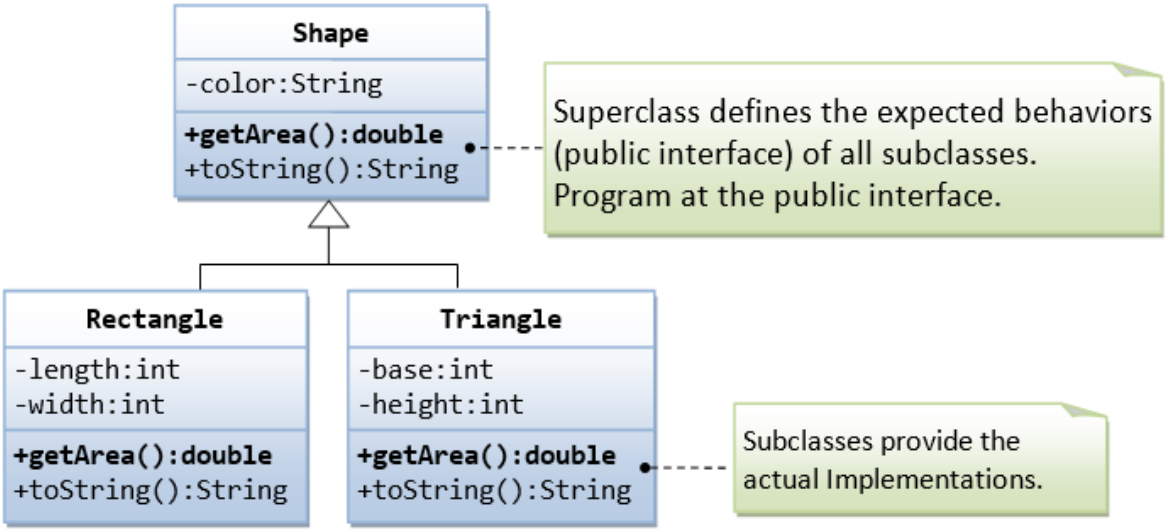
Programming 2

Tutorial 6

Activity 1 (required)

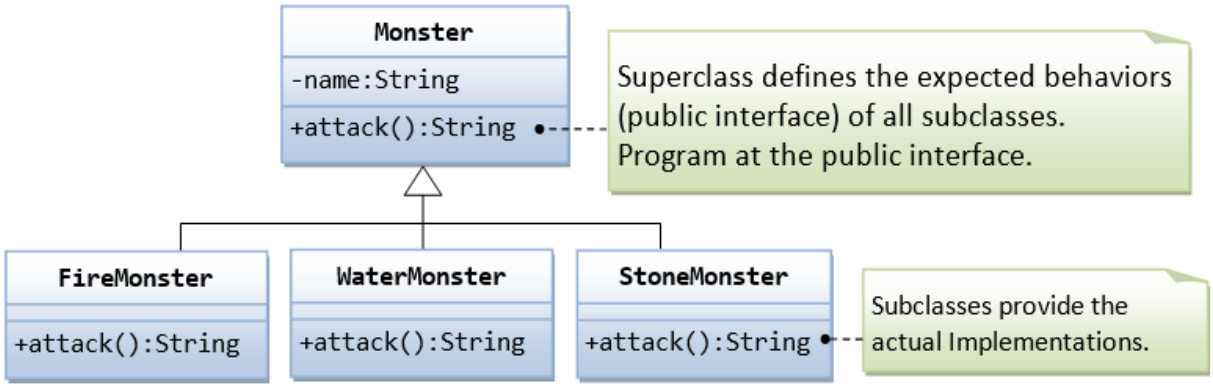
Implement a superclass named `Shape` for a program that deals with various shapes like triangles, rectangles, etc. The `Shape` class should define the public interfaces (or behaviors) shared by all shapes. For instance, all shapes should have a method called `getArea()`, which calculates and returns the area specific to each shape.



Design a test driver program for the `Shape` class and its subclasses (`TestShape.java`). In the `TestShape` class, create some instances of `Rectangle` and `Triangle` for demonstrating polymorphism. Implement necessary upcasting and method calls as shown in Lecture's examples. Ensure that each shape's `toString()` and `getArea()` methods are appropriately invoked.

Activity 2 (required)

In a video game app, we have many types of monsters that can attack. We shall design a superclass called `Monster` and define the method `attack()` in the superclass. The subclasses shall then provides their actual implementation. In the main program, we declare instances of superclass, substituted with actual subclass; and invoke methods defined in the superclass. See the following UML diagram:



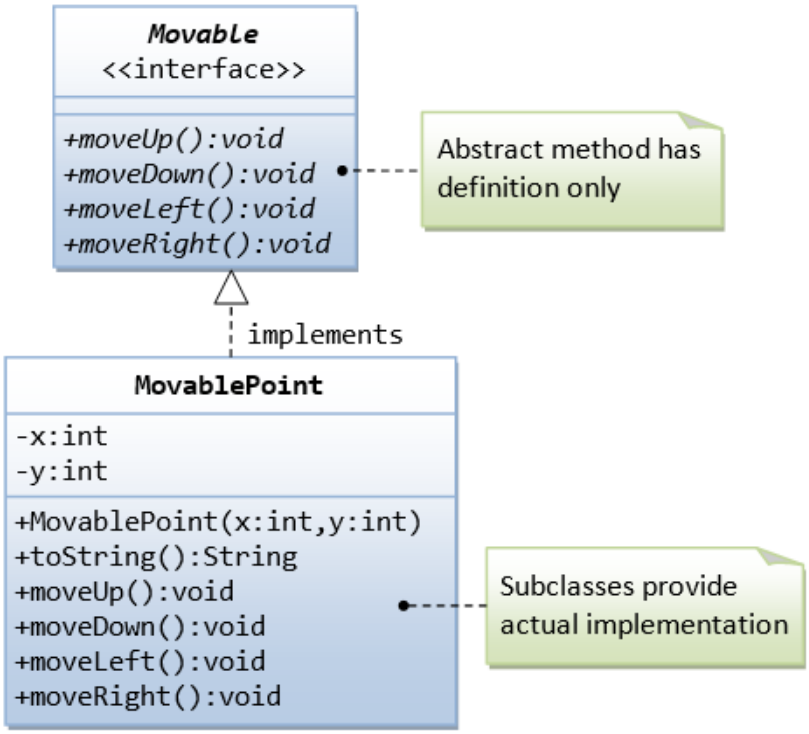
Implement the test driver program (`TestMonster.java`). You are required to declare instances of the superclass, substituted by instances of the subclasses, and invoke relevant methods to demonstrate the use of polymorphism.

Ensure the following functionalities in your implementation:

- Declare references of the superclass (`Monster`), substituting them with instances of the subclasses (`FireMonster`, `WaterMonster`, `StoneMonster`).
- Demonstrate polymorphic behavior by invoking the `attack()` method on these instances, ensuring that the respective subclass' implementation is executed.
- Reassign one of the superclass instances with a new instance of a subclass and verify that the appropriate attack method is called.
- Additionally, handle the scenario where an instance of the superclass is directly created, and ensure an appropriate message is displayed for this case.

Activity 3 (required)

Suppose that our application involves many objects that can move. We could define an interface called `Movable`, containing the signatures of the various movement methods. Implement the application based on the following UML diagram:



`TestMovable.java` (provided on LMS) is a test driver to test the program.

Submission

Submit a **zip** file containing all of your source codes and documents to this tutorial's submission box in the course website on FIT LMS.